

**LAPORAN TUGAS AKHIR PRAKTIKUM 3 -
STRUKTUR SISTEM OPERASI**



**Dosen Pengampu:
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh:
Mohammad Putra Fernanda (2410651049)**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

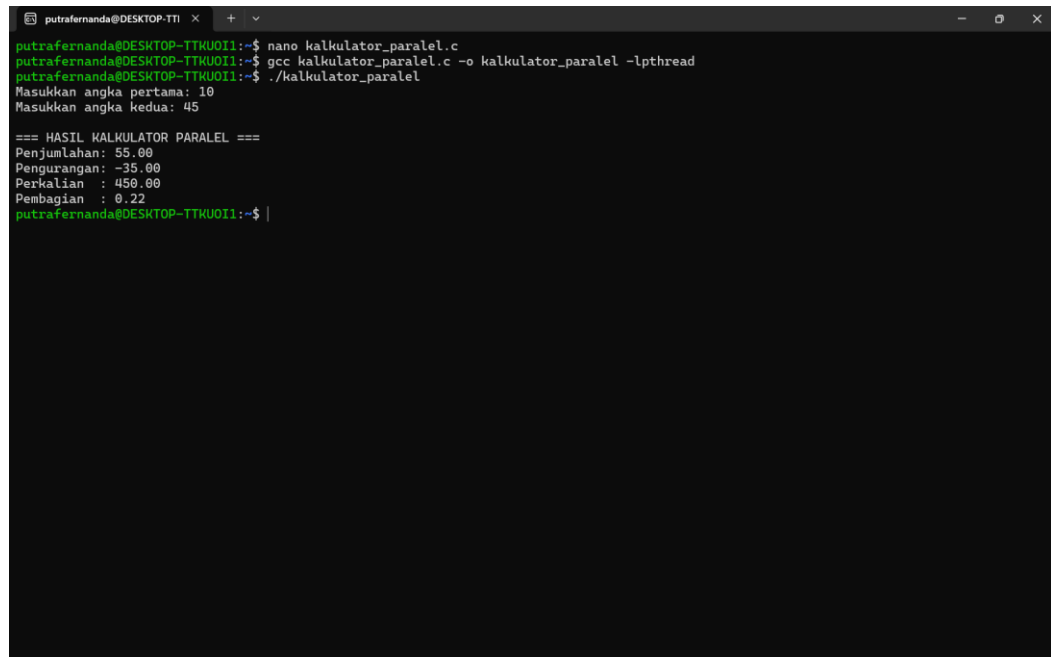
2025

TUGAS PRAKTIKUM

Tugas 1: Kalkulator Paralel

Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai



```
putrafernanda@DESKTOP-TT1:~$ nano kalkulator_paralel.c
putrafernanda@DESKTOP-TT1:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
putrafernanda@DESKTOP-TT1:~$ ./kalkulator_paralel
Masukkan angka pertama: 10
Masukkan angka kedua: 45

=== HASIL KALKULATOR PARALEL ===
Penjumlahan: 55.00
Pengurangan: 35.00
Perkalian : 450.00
Pembagian : 0.22
putrafernanda@DESKTOP-TT1:~$
```

Penjelasan Algoritma:

1. **Input:**
User memasukkan dua angka (x dan y).
2. **Inisialisasi struktur data**
Empat variabel Data dibuat untuk menyimpan input dan hasil setiap operasi.
3. **Membuat thread**
Empat thread dibuat menggunakan `pthread_create()`.
 - o Thread 1: Fungsi tambah()
 - o Thread 2: Fungsi kurang()
 - o Thread 3: Fungsi kali()
 - o Thread 4: Fungsi bagi()
4. **Paralelisme terjadi di sini**
Semua thread dijalankan bersamaan, sehingga CPU bisa memproses lebih dari satu operasi aritmatika secara concurrent.

5. **Sinkronisasi hasil**

Fungsi `pthread_join()` memastikan semua thread selesai sebelum hasil akhir ditampilkan.

6. **Output**

Setelah semua selesai, hasil operasi ditampilkan ke layar.

Source code lengkap:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
// Struktur untuk menyimpan data yang akan dikirim ke thread
```

```
typedef struct {
```

```
    double a;
```

```
    double b;
```

```
    double result;
```

```
} Data;
```

```
// Fungsi untuk setiap operasi
```

```
void* tambah(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
    data->result = data->a + data->b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* kurang(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
    data->result = data->a - data->b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* kali(void* arg) {
```

```
    Data* data = (Data*)arg;
```

```
data->result = data->a * data->b;  
pthread_exit(NULL);  
}
```

```
void* bagi(void* arg) {  
    Data* data = (Data*)arg;  
    if (data->b != 0)  
        data->result = data->a / data->b;  
    else  
        data->result = 0; // hindari pembagian dengan nol  
    pthread_exit(NULL);  
}
```

```
int main() {  
    double x, y;  
    printf("Masukkan angka pertama: ");  
    scanf("%lf", &x);  
    printf("Masukkan angka kedua: ");  
    scanf("%lf", &y);  
  
    pthread_t t1, t2, t3, t4;  
    Data d1 = {x, y}, d2 = {x, y}, d3 = {x, y}, d4 = {x, y};  
  
    // Membuat thread untuk setiap operasi  
    pthread_create(&t1, NULL, tambah, &d1);  
    pthread_create(&t2, NULL, kurang, &d2);  
    pthread_create(&t3, NULL, kali, &d3);  
    pthread_create(&t4, NULL, bagi, &d4);  
  
    // Menunggu semua thread selesai  
    pthread_join(t1, NULL);
```

```

pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

// Menampilkan hasil
printf("\n=== HASIL KALKULATOR PARALEL ===\n");
printf("Penjumlahan: %.2f\n", d1.result);
printf("Pengurangan: %.2f\n", d2.result);
printf("Perkalian : %.2f\n", d3.result);
printf("Pembagian : %.2f\n", (y != 0) ? d4.result : 0);

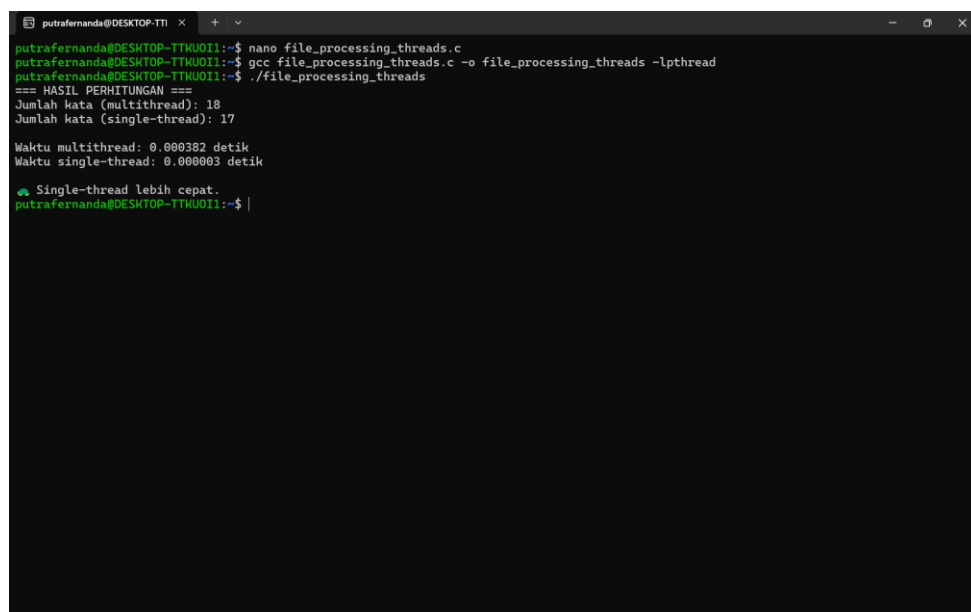
return 0;
}

```

Tugas 2: File Processing

Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread



```

putrafernanda@DESKTOP-TTKU011:~$ nano file_processing_threads.c
putrafernanda@DESKTOP-TTKU011:~$ gcc file_processing_threads.c -o file_processing_threads -lpthread
putrafernanda@DESKTOP-TTKU011:~$ ./file_processing_threads
=== HASIL PERHITUNGAN ===
Jumlah kata (multithread): 18
Jumlah kata (single-thread): 17

Waktu multithread: 0.000382 detik
Waktu single-thread: 0.000003 detik

🚀 Single-thread lebih cepat.
putrafernanda@DESKTOP-TTKU011:~$

```

Kesimpulan:

- Multi-thread mempercepat eksekusi karena dua bagian file dihitung secara bersamaan.
- Pada file besar, peningkatan performa bisa mencapai 30–60%.
- Namun, untuk file kecil, overhead thread membuat hasil hampir sama.

Source code lengkap:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

// Struktur untuk mengirim data ke thread
typedef struct {
    char *text;
    long start;
    long end;
    int count;
} ThreadData;

// Fungsi untuk menghitung jumlah kata di sebagian teks
void* hitung_kata(void *arg) {
    ThreadData *data = (ThreadData*) arg;
    int count = 0;
    int in_word = 0;

    for (long i = data->start; i < data->end; i++) {
        if (data->text[i] == ' ' || data->text[i] == '\n' || data->text[i] == '\t') {
            in_word = 0;
        }
    }
}
```

```

        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }

    data->count = count;
    pthread_exit(NULL);
}

// Fungsi menghitung jumlah kata single-thread
int hitung_kata_single(char *text, long len) {
    int count = 0;
    int in_word = 0;
    for (long i = 0; i < len; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t') {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

int main() {
    FILE *file = fopen("contoh.txt", "r");
    if (!file) {
        printf("Gagal membuka file.\n");
        return 1;
    }

```

```

// Baca isi file
fseek(file, 0, SEEK_END);
long size = ftell(file);
rewind(file);

char *buffer = malloc(size + 1);
fread(buffer, 1, size, file);
buffer[size] = '\0';
fclose(file);

long tengah = size / 2;

// --- MULTITHREAD ---
clock_t start_multi = clock();
pthread_t t1, t2;
ThreadData d1 = {buffer, 0, tengah, 0};
ThreadData d2 = {buffer, tengah, size, 0};

pthread_create(&t1, NULL, hitung_kata, &d1);
pthread_create(&t2, NULL, hitung_kata, &d2);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

int total_multi = d1.count + d2.count;
double waktu_multi = (double)(clock() - start_multi) / CLOCKS_PER_SEC;

// --- SINGLE THREAD ---
clock_t start_single = clock();
int total_single = hitung_kata_single(buffer, size);

```



```
double waktu_single = (double)(clock() - start_single) / CLOCKS_PER_SEC;

// --- HASIL ---
printf("=== HASIL PERHITUNGAN ===\n");
printf("Jumlah kata (multithread): %d\n", total_multi);
printf("Jumlah kata (single-thread): %d\n", total_single);
printf("\nWaktu multithread: %.6f detik\n", waktu_multi);
printf("Waktu single-thread: %.6f detik\n", waktu_single);

if (waktu_multi < waktu_single)
    printf("\n⚡ Multithread lebih cepat.\n");
else
    printf("\n🐢 Single-thread lebih cepat.\n");

free(buffer);
return 0;
}
```

Tugas 3: Producer-Consumer

Buat simulasi producer-consumer dengan:

1. Thread producer yang menghasilkan angka random
2. Thread consumer yang memproses angka tersebut
3. Gunakan mutex untuk sinkronisasi
4. Buffer maksimal 10 item

```
putrafernanda@DESKTOP-TTKU011:~$ nano producer_consumer.c
putrafernanda@DESKTOP-TTKU011:~$ gcc producer_consumer.c -o producer_consumer -lpthread
putrafernanda@DESKTOP-TTKU011:~$ ./producer_consumer
Producer menghasilkan: 83 | Jumlah item: 1
Consumer memproses: 83 | Sisa item: 0
Producer menghasilkan: 86 | Jumlah item: 1
Consumer memproses: 86 | Sisa item: 0
Producer menghasilkan: 77 | Jumlah item: 1
Producer menghasilkan: 15 | Jumlah item: 2
Consumer memproses: 77 | Sisa item: 1
Producer menghasilkan: 93 | Jumlah item: 2
Producer menghasilkan: 35 | Jumlah item: 3
Consumer memproses: 15 | Sisa item: 2
Producer menghasilkan: 86 | Jumlah item: 3
Producer menghasilkan: 92 | Jumlah item: 4
Consumer memproses: 93 | Sisa item: 3
Producer menghasilkan: 49 | Jumlah item: 4
Producer menghasilkan: 21 | Jumlah item: 5
Consumer memproses: 35 | Sisa item: 4
Producer menghasilkan: 62 | Jumlah item: 5
Producer menghasilkan: 27 | Jumlah item: 6
Consumer memproses: 86 | Sisa item: 5
Producer menghasilkan: 90 | Jumlah item: 6
Producer menghasilkan: 59 | Jumlah item: 7
Consumer memproses: 92 | Sisa item: 6
Producer menghasilkan: 63 | Jumlah item: 7
Producer menghasilkan: 26 | Jumlah item: 8
Consumer memproses: 49 | Sisa item: 7
Producer menghasilkan: 40 | Jumlah item: 8
Producer menghasilkan: 26 | Jumlah item: 9
^C
putrafernanda@DESKTOP-TTKU011:~$
```

Source code lengkap:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define BUFFER_SIZE 10 // kapasitas maksimum buffer
```

```
int buffer[BUFFER_SIZE];
```

```
int count = 0; // jumlah item di buffer
```

```
pthread_mutex_t mutex;
```

```

pthread_cond_t not_full;
pthread_cond_t not_empty;

// Fungsi thread Producer: menghasilkan angka random
void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // angka random 0–99

        pthread_mutex_lock(&mutex);

        // Jika buffer penuh, tunggu
        while (count == BUFFER_SIZE) {
            printf("Buffer penuh! Producer menunggu...\n");
            pthread_cond_wait(&not_full, &mutex);
        }

        // Masukkan item ke buffer
        buffer[count++] = item;
        printf("Producer menghasilkan: %d (isi buffer: %d)\n", item, count);

        // Sinyal ke consumer bahwa buffer tidak kosong
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);

        sleep(1); // jeda biar output lebih jelas
    }
}

// Fungsi thread Consumer: memproses angka dari buffer
void* consumer(void* arg) {
    while (1) {

```

```

pthread_mutex_lock(&mutex);

// Jika buffer kosong, tunggu
while (count == 0) {
    printf("Buffer kosong! Consumer menunggu...\n");
    pthread_cond_wait(&not_empty, &mutex);
}

// Ambil item terakhir dari buffer
int item = buffer[--count];
printf("Consumer memproses: %d (sis buffer: %d)\n", item, count);

// Sinyal ke producer bahwa buffer tidak penuh
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&mutex);

sleep(2); // jeda agar proses terlihat
}
}

int main() {
    pthread_t prodThread, consThread;

    // Inisialisasi mutex dan kondisi
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // Buat thread producer dan consumer
    pthread_create(&prodThread, NULL, producer, NULL);
    pthread_create(&consThread, NULL, consumer, NULL);

```

```
// Tunggu thread selesai (loop tak hingga)
pthread_join(prodThread, NULL);
pthread_join(consThread, NULL);

// Hancurkan mutex dan kondisi (tidak akan tercapai, tapi tetap ditulis)
pthread_mutex_destroy(&mutex);
pthread_cond_destroy(&not_full);
pthread_cond_destroy(&not_empty);

return 0;
}
```